

HMS Veterinary SaaS

Final Deployment Guide

Docker · AWS EC2 · MongoDB Atlas · GitHub Actions

Ubuntu 24.04 · ap-south-1 Mumbai · Testing → Production

- React 19
- Node.js
- MongoDB
- Docker
- AWS ECR
- GitHub Actions

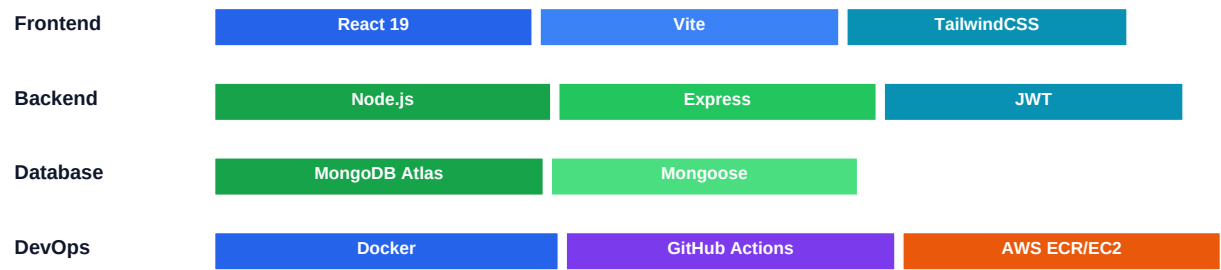
ACTUAL DEPLOYMENT VALUES	
EC2 Public IP	13.232.125.254
EC2 OS	Ubuntu Server 24.04 LTS
EC2 Type	t3.small (2 vCPU, 2 GB RAM)
ECR Registry	232261469028.dkr.ecr.ap-south-1.amazonaws.com
MongoDB	cluster0.lponosn.mongodb.net / hms_db
Frontend	http://13.232.125.254
Backend API	http://13.232.125.254:5000
SSH Key	hms-Key.pem
SSH User	ubuntu

Table of Contents

1.	Project Overview & Tech Stack
2.	System Architecture Diagram
3.	CI/CD Pipeline Flow
4.	User Roles & Access Map
5.	Application Routes Reference
Step 1	MongoDB Atlas Setup
Step 2	AWS IAM & ECR Setup
Step 3	EC2 Instance Launch & Bootstrap
Step 4	Environment Configuration
Step 5	GitHub Actions Secrets
Step 6	Project Files (all config files)
Step 7	Deploy & Verify
—	Ongoing Workflow
—	Troubleshooting Guide
—	Production Checklist

1. Project Overview & Tech Stack

WebArclight Veterinary HMS is a multi-tenant SaaS platform for veterinary clinics. It supports multiple user roles, clinic management, pet records, appointments, lab results, grooming, kennel management, and payment processing.



Technology Stack Overview
Fig 1 — Technology stack layers

Layer	Technology	Version/Details
Frontend	React + Vite + TailwindCSS	React 19, Vite 8, Tailwind 4
Backend	Node.js + Express	Node 20 Alpine, Express 4
Database	MongoDB Atlas	Cluster0, M0 Free, ap-south-1
File Storage	AWS S3 + Cloudinary	multiplex-s3, Cloudinary CDN
Payments	Razorpay	razorpay 2.9.2
Email	Nodemailer + Gmail SMTP	smartvetdr@gmail.com
Containerization	Docker + Docker Compose	Node 20 Alpine + nginx 1.27 Alpine
Image Registry	AWS ECR	ap-south-1, Account: 232261469028
Hosting	AWS EC2	Ubuntu 24.04 LTS, t3.small, 13.232.125.254
CI/CD	GitHub Actions	Auto-deploy on push to main/develop
Monitoring	Docker healthchecks	curl-based, 30s interval

2. System Architecture Diagram

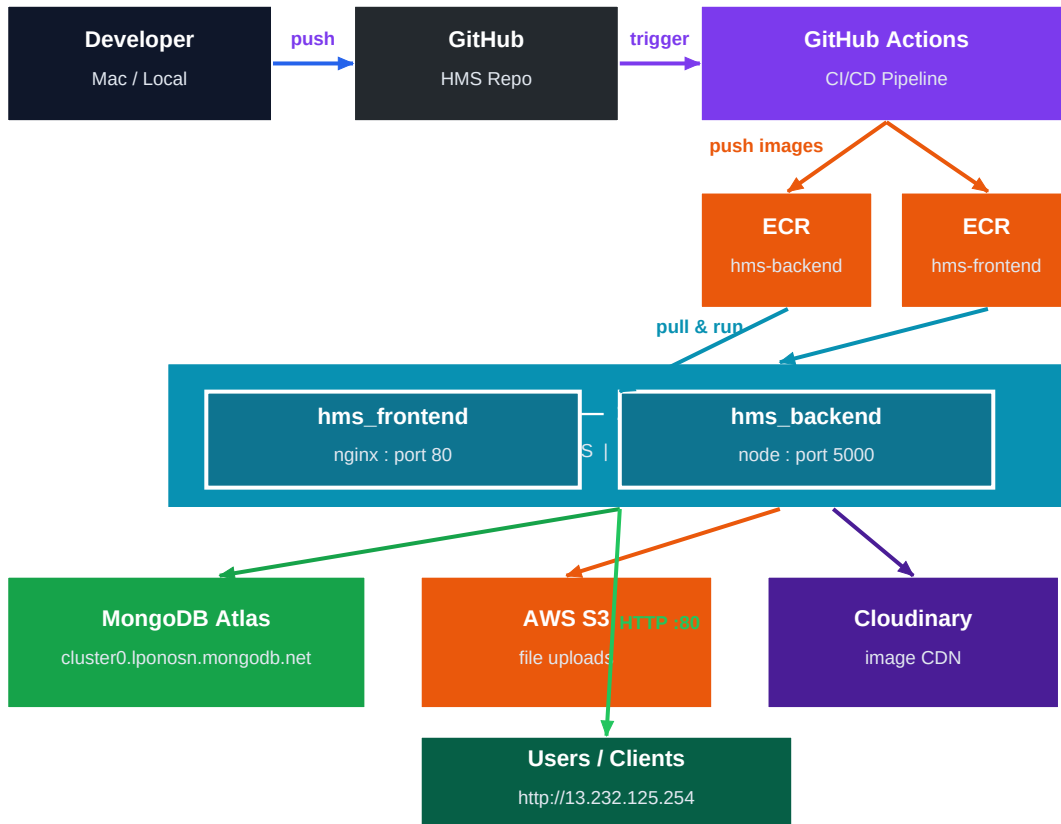


Fig 2 — Full system architecture: Developer → GitHub → Actions → ECR → EC2 → MongoDB Atlas

The deployment is fully automated. A git push triggers GitHub Actions which builds Docker images, pushes them to AWS ECR, then SSHs into the EC2 instance and runs docker compose to update containers. MongoDB Atlas is hosted separately in the same AWS Mumbai region for low latency.

3. CI/CD Pipeline Flow



Total deploy time: ~4-6 min (first) / ~90 sec (subsequent)

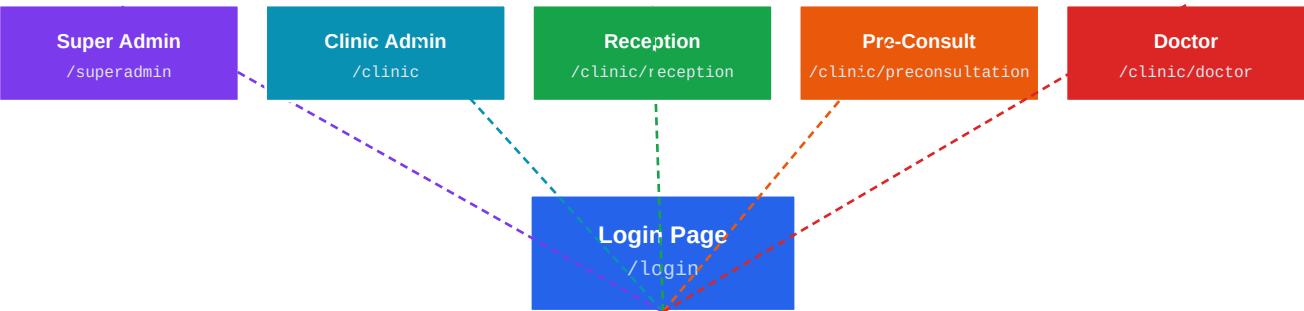
Fig 3 — GitHub Actions pipeline steps and timing

The pipeline has two jobs:

- build-and-push: Checks out code, authenticates with ECR, builds both Docker images tagged with Git commit SHA, pushes to ECR
- deploy: Copies docker-compose.yml to EC2 via SCP, SSHs in, pulls new images, runs docker compose up -d, prunes old images

Trigger	Branch	Duration	Result
git push	main or develop	4-6 min (first) / 90 sec (subsequent)	Both containers redeployed
Manual	Any	~90 sec	docker compose up -d on EC2

4. User Roles & Access Map



All role dashboards are protected — login redirects to appropriate dashboard

Fig 4 — User roles and their entry points after login

Role	Entry URL	Access Level	Description
Super Admin	/superadmin	Full platform	Manage all clinics, plans, vets, reports
Clinic Admin	/clinic	Clinic scope	Staff, doctors, lab, groomer, kennel, settings
Reception	/clinic/reception	Reception scope	Pet registration, existing customers, history
Pre-Consultation	/clinic/preconsultation	Pre-consult scope	Pending, completed, history pets
Doctor	/clinic/doctor	Doctor scope	Pending, completed, history patients
Public	/ and /login	No auth	Home, about, contact, login

5. Application Routes Reference

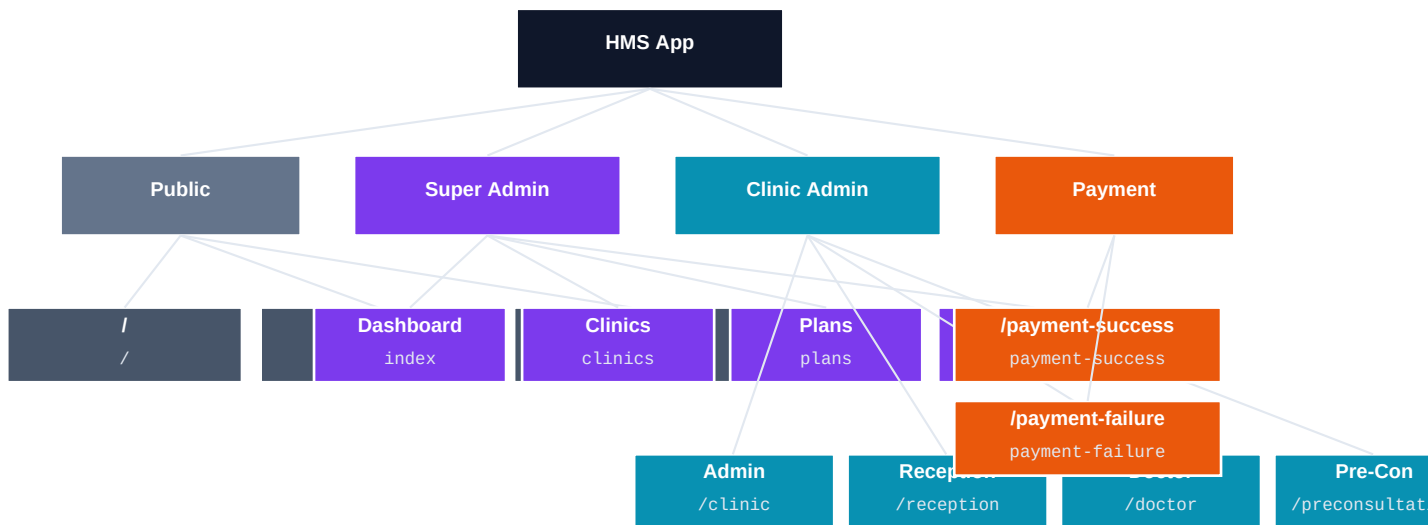


Fig 5 — Application route tree

All Available Routes

Route	Page	Auth Required	Role
http://13.232.125.254/	Home	No	Public
http://13.232.125.254/login	Login	No	Public
http://13.232.125.254/about	About	No	Public
http://13.232.125.254/contact	Contact	No	Public
http://13.232.125.254/superadmin	SA Dashboard	Yes	SUPER_ADMIN
http://13.232.125.254/superadmin/clinics	Clinics	Yes	SUPER_ADMIN
http://13.232.125.254/superadmin/veterinarian	Vets	Yes	SUPER_ADMIN
http://13.232.125.254/superadmin/plans	Plans	Yes	SUPER_ADMIN
http://13.232.125.254/superadmin/reports	Reports	Yes	SUPER_ADMIN
http://13.232.125.254/superadmin/verification	Verification	Yes	SUPER_ADMIN
http://13.232.125.254/clinic	Clinic Dashboard	Yes	CLINIC_ADMIN
http://13.232.125.254/clinic/doctors	Doctors	Yes	CLINIC_ADMIN
http://13.232.125.254/clinic/staff	Staff	Yes	CLINIC_ADMIN
http://13.232.125.254/clinic/lab	Lab	Yes	CLINIC_ADMIN
http://13.232.125.254/clinic/groomer	Groomer	Yes	CLINIC_ADMIN
http://13.232.125.254/clinic/kennel	Kennel	Yes	CLINIC_ADMIN

http://13.232.125.254/clinic/reports	Reports	Yes	CLINIC_ADMIN
http://13.232.125.254/clinic/settings	Settings	Yes	CLINIC_ADMIN
http://13.232.125.254/clinic/reception	Reception	Yes	RECEPTION
http://13.232.125.254/clinic/reception/new-registration	New Pet	Yes	RECEPTION
http://13.232.125.254/clinic/reception/existing-customer	Existing	Yes	RECEPTION
http://13.232.125.254/clinic/preconsultation	Pre-Consult	Yes	STAFF
http://13.232.125.254/clinic/doctor	Doctor	Yes	DOCTOR
http://13.232.125.254/payment	Payment	No	Public

1 MongoDB Atlas Setup

ACTUAL DEPLOYMENT VALUES

Organization	MAYANK's Org - 2025-08-28
Project	HMS
Cluster	Cluster0 (M0 Free Tier)
Region	AWS / Mumbai (ap-south-1)
DB User	mayank_db_user (used in backend .env)
DB User 2	hms_user (HMS Testing Server)
Whitelisted IP	13.232.125.254/32 – Active
Database Name	hms_db

Create cluster

- <https://cloud.mongodb.com> -> Build a Database -> M0 Free Tier
- AWS, region ap-south-1 (Mumbai) -> Name: Cluster0 -> Create

Create database user

- Security -> Database & Network Access -> Database Access -> + Add New Database User
- Method: Password | Username: hms_user | Autogenerate password -> Copy it
- Built-in Role: Atlas admin -> Add User

Whitelist EC2 IP

- Network Access -> + Add IP Address -> 13.232.125.254/32
- Comment: HMS_EC2_IPAddress -> Confirm

■ Remove 0.0.0.0/0 entry before going to production.

Connection string format

```
mongodb+srv://mayank_db_user:@cluster0.lponosn.mongodb.net/hms_db?retryWrites=true&w=majority
```

Test connection from EC2

```
# Install mongosh on Ubuntu 24.04
wget -qO- https://www.mongodb.org/static/pgp/server-7.0.asc | sudo apt-key add -
echo "deb [ arch=amd64,arm64 ] https://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/7.0 multiverse" \
| sudo tee /etc/apt/sources.list.d/mongodb-org-7.0.list
sudo apt-get update -y && sudo apt-get install -y mongodb-mongosh

mongosh "mongodb+srv://mayank_db_user:@cluster0.lponosn.mongodb.net/hms_db"
# Success: Atlas atlas-xxxxx-shard-0 [primary] hms_db>
```

2

AWS IAM & ECR Setup

ACTUAL DEPLOYMENT VALUES	
IAM User	<code>hms-cicd-user</code> (<code>AmazonEC2ContainerRegistryPowerUser</code>)
EC2 Role	<code>Hms-ec2-ecr-role</code> (<code>AmazonEC2ContainerRegistryReadOnly</code>)
ECR Registry	<code>232261469028.dkr.ecr.ap-south-1.amazonaws.com</code>
Backend Repo	<code>232261469028.dkr.ecr.ap-south-1.amazonaws.com/hms-backend</code>
Frontend Repo	<code>232261469028.dkr.ecr.ap-south-1.amazonaws.com/hms-frontend</code>

Create IAM user for GitHub Actions

- IAM -> Users -> Create User -> Username: `hms-cicd-user`
- Attach policy: `AmazonEC2ContainerRegistryPowerUser`
- Security credentials -> Create access key -> Application outside AWS -> Download CSV

■ Secret key only shown once — save it immediately.

Create EC2 IAM Role

- IAM -> Roles -> Create Role -> AWS service -> EC2
- Attach: `AmazonEC2ContainerRegistryReadOnly` -> Name: `Hms-ec2-ecr-role` -> Create

Create ECR repositories

```
aws configure # use hms-cicd-user credentials, region: ap-south-1
aws ecr create-repository --repository-name hms-backend --region ap-south-1
aws ecr create-repository --repository-name hms-frontend --region ap-south-1
```

3 EC2 Instance Launch & Bootstrap

ACTUAL DEPLOYMENT VALUES

AMI	Ubuntu Server 24.04 LTS ami-006f82a1d5a27da54
Instance Type	t3.small (2 vCPU, 2 GB RAM)
Public IP	13.232.125.254
Key Pair	hms-Key.pem at /Volumes/Testing/hms/hms-Key.pem
IAM Profile	Hms-ec2-ecr-role
SSH Command	ssh -i /Volumes/Testing/hms/hms-Key.pem ubuntu@13.232.125.254

Security Group

Port	Source	Purpose
22	0.0.0.0/0	SSH access
80	0.0.0.0/0	Frontend (React app via nginx)
5000	0.0.0.0/0	Backend Express API

- Auto-assign Public IP must be ENABLED.
- Username is 'ubuntu' NOT 'ec2-user' — this is Ubuntu 24.04.

Bootstrap commands (Ubuntu 24.04 — use apt NOT yum)

```
sudo apt-get update -y
sudo apt-get install -y docker.io
sudo systemctl start docker && sudo systemctl enable docker
sudo usermod -aG docker ubuntu

sudo mkdir -p /usr/local/lib/docker/cli-plugins
sudo curl -SL https://github.com/docker/compose/releases/latest/download/docker-compose-linux-x86_64 \
-o /usr/local/lib/docker/cli-plugins/docker-compose
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose

sudo snap install aws-cli --classic # apt package not available on Ubuntu 24.04

mkdir -p ~/hms && mkdir -p ~/hms/backend

exit # re-login for docker group
```

4 Environment Configuration

Copy backend .env to EC2

```
scp -i /Volumes/Testing/hms/hms-Key.pem \  
/Volumes/Testing/hms/HMS_Milestone-2/backend/.env \  
ubuntu@13.232.125.254:~/hms/backend/.env
```

■ File must be at ~/hms/backend/.env — docker-compose.yml looks for it there.

Complete backend .env

```
PORT=5000  
NODE_ENV=production  
  
MONGO_URI=mongodb+srv://mayank_db_user:ZFwhxsCjU5JxL4uR@cluster0.lponosn.mongodb.net/hms_db?retryWrites  
=true&w;=majority  
  
JWT_SECRET=hms_super_secret_jwt_2026_webarclight_xK9mP2qR  
SUPER_ADMIN_MOBILE=7500447866  
  
FRONTEND_URL=http://13.232.125.254  
  
RAZORPAY_KEY_ID=  
RAZORPAY_KEY_SECRET=  
AWS_REGION=  
AWS_ACCESS_KEY_ID=  
AWS_SECRET_ACCESS_KEY=  
S3_BUCKET=  
  
SMTP_HOST=smtp.gmail.com  
SMTP_PORT=587  
SMTP_EMAIL=smartvetdr@gmail.com  
SMTP_PASSWORD=agpyyombwixmkpsq  
FROM_NAME="HMS Platform"  
FROM_EMAIL=smartvetdr@gmail.com  
  
CLOUDINARY_CLOUD_NAME=dulxydos7  
CLOUDINARY_API_KEY=224328736491543  
CLOUDINARY_API_SECRET=0YoR8Db3YRo7tKZAQxMJ7lI6ao
```

Frontend .env (local — baked into Docker image at build time)

```
VITE_BACKEND_URL=http://13.232.125.254:5000/api/v1
```

■ Frontend env is not copied to EC2 — it is injected via VITE_API_URL build-arg in GitHub Actions.

5 GitHub Actions Secrets

GitHub repo -> Settings -> Secrets and variables -> Actions -> New repository secret

Secret Name	Value
AWS_ACCESS_KEY_ID	hms-cicd-user access key
AWS_SECRET_ACCESS_KEY	hms-cicd-user secret key
AWS_REGION	ap-south-1
ECR_REGISTRY	232261469028.dkr.ecr.ap-south-1.amazonaws.com
EC2_HOST	13.232.125.254
EC2_USER	ubuntu
EC2_SSH_KEY	Full contents of hms-Key.pem (include BEGIN/END lines)
VITE_API_URL	http://13.232.125.254:5000

```
cat /Volumes/Testing/hms/hms-Key.pem # copy this entire output into EC2_SSH_KEY
```

6 Project Files

```
HMS_Milestone-2/
|-- backend/
| |-- Dockerfile <- Node.js container
| |-- .env <- Production secrets (DO NOT COMMIT)
| `-- .env.example <- Template for team
|-- frontend/main-app/
| |-- Dockerfile <- Vite build + nginx
| |-- nginx.conf <- SPA routing + caching
| `-- .env <- VITE_BACKEND_URL
|-- docker-compose.yml <- Orchestrates services
|-- .github/workflows/
| `-- deploy-test.yml <- CI/CD pipeline
|-- DEPLOYMENT_GUIDE.md <- This guide (markdown)
`-- HMS_Final_Deployment_Guide.pdf <- This document
```

backend/Dockerfile

```
FROM node:20-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production

FROM node:20-alpine AS runner
WORKDIR /app
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN chown -R appuser:appgroup /app
USER appuser
EXPOSE 5000
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
CMD wget -qO- http://localhost:5000/health || exit 1
CMD ["node", "src/index.js"]
```

frontend/main-app/Dockerfile

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
ARG VITE_API_URL=http://localhost:5000
ENV VITE_API_URL=$VITE_API_URL
RUN npm run build

FROM nginx:1.27-alpine AS runner
RUN apk add --no-cache curl
```

```
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
HEALTHCHECK --interval=30s --timeout=5s --start-period=5s --retries=3 \
CMD curl -f http://localhost:80 || exit 1
CMD ["nginx", "-g", "daemon off;"]
```

frontend/main-app/nginx.conf

```
server {
  listen 80;
  server_name _;
  root /usr/share/nginx/html;
  index index.html;
  gzip on;
  gzip_types text/plain text/css application/json application/javascript
  text/xml application/xml image/svg+xml;
  location / { try_files $uri $uri/ /index.html; }
  location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff2?)$ {
    expires 1y;
    add_header Cache-Control "public, immutable";
  }
  location = /index.html {
    add_header Cache-Control "no-cache, no-store, must-revalidate";
  }
}
```

docker-compose.yml

```
services:
  backend:
    image: ${ECR_REGISTRY:-hms}/hms-backend:${IMAGE_TAG:-latest}
    build:
      context: ./backend
    container_name: hms_backend
    restart: unless-stopped
    ports: ["5000:5000"]
    env_file: [./backend/.env]
    environment:
      NODE_ENV: ${NODE_ENV:-production}
    healthcheck:
      test: ["CMD", "wget", "-qO-", "http://localhost:5000/health"]
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 15s
  frontend:
    image: ${ECR_REGISTRY:-hms}/hms-frontend:${IMAGE_TAG:-latest}
    build:
      context: ./frontend/main-app
    args:
      VITE_API_URL: ${VITE_API_URL:-http://localhost:5000}
    container_name: hms_frontend
    restart: unless-stopped
    ports: ["80:80"]
    depends_on:
```

```
backend:
condition: service_healthy
healthcheck:
test: ["CMD", "curl", "-f", "http://localhost:80"]
interval: 30s
timeout: 5s
retries: 3
start_period: 10s
```

[.github/workflows/deploy-test.yml](#)

```
name: Deploy to AWS EC2 (Testing)
on:
push:
branches: [main, develop]

jobs:
build-and-push:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- uses: aws-actions/configure-aws-credentials@v4
with:
aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
aws-region: ${ secrets.AWS_REGION }
- uses: aws-actions/amazon-ecr-login@v2
- run: |
docker build -t ${ secrets.ECR_REGISTRY }/hms-backend:$GITHUB_SHA ./backend
docker push ${ secrets.ECR_REGISTRY }/hms-backend:$GITHUB_SHA
- run: |
docker build --build-arg VITE_API_URL=${ secrets.VITE_API_URL } \
-t ${ secrets.ECR_REGISTRY }/hms-frontend:$GITHUB_SHA ./frontend/main-app
docker push ${ secrets.ECR_REGISTRY }/hms-frontend:$GITHUB_SHA

deploy:
needs: build-and-push
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- uses: appleboy/scp-action@v0.1.7
with:
host: ${ secrets.EC2_HOST }
username: ${ secrets.EC2_USER }
key: ${ secrets.EC2_SSH_KEY }
source: docker-compose.yml
target: /home/${ secrets.EC2_USER }/hms
- uses: appleboy/ssh-action@v1
with:
host: ${ secrets.EC2_HOST }
username: ${ secrets.EC2_USER }
```

```
key: ${ secrets.EC2_SSH_KEY }}
script: |
cd ~/hms
aws ecr get-login-password --region ${ secrets.AWS_REGION }} \
| docker login --username AWS --password-stdin ${ secrets.ECR_REGISTRY }}
export ECR_REGISTRY=${ secrets.ECR_REGISTRY }}
export IMAGE_TAG=${ github.sha }}
docker compose pull && docker compose up -d --remove-orphans
docker image prune -f
```

7

Deploy & Verify

Trigger deploy

```
cd /Volumes/Testing/hms/HMS_Milestone-2
git add .
git commit -m "chore: add Docker and CI/CD configuration"
git push origin main
```

Verify on EC2

```
ssh -i /Volumes/Testing/hms/hms-Key.pem ubuntu@13.232.125.254

docker ps
# Expected:
# hms_frontend Up X minutes (healthy) 0.0.0.0:80->80/tcp
# hms_backend Up X minutes (healthy) 0.0.0.0:5000->5000/tcp

curl http://localhost:5000/health
# {"status":"Platform is online and running."}

docker logs hms_backend 2>&1 | grep -i "mongo\|connected"
# MongoDB Connected: cluster0.lponosn.mongodb.net
```

Access URLs

Service	URL	Status
Frontend App	http://13.232.125.254	■ Live
Backend Health	http://13.232.125.254:5000/health	■ Live
API Base	http://13.232.125.254:5000/api/v1/	■ Live
Login Page	http://13.232.125.254/login	■ Live
Super Admin	http://13.232.125.254/superadmin	■ Live (auth required)
Clinic Admin	http://13.232.125.254/clinic	■ Live (auth required)

Ongoing Workflow

```
git push origin main/develop
--> GitHub Actions builds & deploys (~90 sec)
--> Live at http://13.232.125.254
```

Update .env on EC2

```
ssh -i /Volumes/Testing/hms/hms-Key.pem ubuntu@13.232.125.254
nano ~/hms/backend/.env
docker compose restart backend
```

Manual deploy from local

```
export ECR_REGISTRY=232261469028.dkr.ecr.ap-south-1.amazonaws.com
aws ecr get-login-password --region ap-south-1 \
| docker login --username AWS --password-stdin $ECR_REGISTRY
ECR_REGISTRY=$ECR_REGISTRY IMAGE_TAG=latest docker compose up -d
```

Useful commands

```
docker ps # container status
docker stats # live CPU/RAM
docker logs hms_backend -f # stream backend logs
docker logs hms_frontend -f # stream frontend logs
docker compose down && docker compose up -d # full restart
docker image prune -f # clean old images
```

Troubleshooting Guide

Problem	Cause	Fix
yum: command not found	Ubuntu 24.04 uses apt	sudo apt-get install instead of yum
awscli not available via apt	Not in Ubuntu 24.04 apt	sudo snap install aws-cli --classic
Container unhealthy (frontend)	wget missing in nginx alpine	use curl -f or install curl via apk add --no-cache curl
env file not found	Wrong path on EC2	File must be at ~/hms/backend/.env not ~/hms/.env
MongoDB connection refused	IP not whitelisted	Atlas -> Network Access -> add 13.232.125.254/32
CORS error in browser	FRONTEND_URL wrong	Set FRONTEND_URL=http://13.232.125.254 in .env
Blank frontend page	VITE_API_URL wrong	Set VITE_API_URL=http://13.232.125.254:5000 in GitHub Secrets
ECR pull access denied	Not logged in	Run aws ecr get-login-password and docker login first
GitHub Actions SSH fails	Wrong user	EC2_USER must be 'ubuntu' not 'ec2-user'
Port 5000 unreachable	Security Group	Add inbound TCP 5000 from 0.0.0.0/0 in EC2 Security Group

Production Checklist

Complete these before going live with real clients:

Security

- Remove 0.0.0.0/0 from MongoDB Atlas Network Access
- Restrict EC2 port 22 to your office IP only
- Change JWT_SECRET to 64-char cryptographically random string
- Set NODE_ENV=production (already done)
- Rotate SMTP app password periodically

Infrastructure

- Upgrade MongoDB Atlas from M0 to M10+ for production load
- Upgrade EC2 from t3.small to t3.medium or larger
- Enable MongoDB Atlas automated backups
- Add Elastic IP to EC2 so IP doesn't change on restart
- Set up CloudWatch alarms for CPU, memory, disk

Domain & SSL

- Point domain/subdomain to EC2 IP (A record in DNS)
- Install SSL certificate (use Certbot / Let's Encrypt or ACM)
- Update FRONTEND_URL to https://yourdomain.com
- Update VITE_API_URL to https://api.yourdomain.com
- Add HTTPS (port 443) to EC2 Security Group

Features

- Fill in RAZORPAY_KEY_ID and RAZORPAY_KEY_SECRET
- Fill in AWS_ACCESS_KEY_ID, S3_BUCKET for file uploads
- Test all user role logins end-to-end
- Test payment flow with Razorpay test keys
- Test email notifications via SMTP

WebArclight Veterinary HMS

Final Deployment Guide · v2.0 · June 2026

EC2: 13.232.125.254 · MongoDB: cluster0.lponosn.mongodb.net · Region: ap-south-1

React 19 · Node.js · MongoDB Atlas · Docker · AWS ECR + EC2 · GitHub Actions